

Lab 3: Multi-Source Retail Sales Data Integration and Analysis

Domain: Retail Analytics
Dataset: UCI Online Retail Dataset

Objective

The objective of this experiment is to perform end-to-end retail data processing using the UCI Online Retail dataset. The original dataset contains transaction, product and customer-related information in a single Excel file. To simulate a multi-source retail environment, the dataset is logically divided into three data sources:

1. Transaction data
2. Product data
3. Customer data

Installing Required Libraries

```
install.packages(
  c("readxl", "dplyr", "DBI", "RSQLite"),
  repos = "https://cloud.r-project.org"
)
```

```
library(readxl)
library(dplyr)
library(tidyr)
library(DBI)
library(RSQLite)
library(ggplot2)
library(lubridate)
```

1. Import the UCI Online Retail Dataset

The UCI Online Retail dataset is provided as an Excel file named `Online Retail.xlsx`. Unlike the original problem statement, where transaction, product and customer information are stored separately, the supplied dataset contains all required information in a single Excel file. Therefore, the dataset will first be imported into R and then logically separated into transaction, product and customer tables.

```
file_path <- "/content/Online Retail.xlsx"

retail_raw <- read_excel(file_path)

head(retail_raw)
```

A tibble: 6 × 8								
InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	
<chr>	<chr>	<chr>	<dbl>	<dtm>	<dbl>	<dbl>	<chr>	
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850	United Kingdom	
536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850	United Kingdom	
536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850	United Kingdom	
536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850	United Kingdom	
536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850	United Kingdom	
536365	22752	SET 7 BABUSHKA NESTING BOXES	2	2010-12-01 08:26:00	7.65	17850	United Kingdom	

Dataset Inspection

- The dataset is inspected to understand:
- Number of records
 - Number of attributes
 - Column names
 - Data types
 - Missing values
 - Duplicate records

```
dim(etail_raw)

541909 · 8
```

```
names(etail_raw)

'InvoiceNo' · 'StockCode' · 'Description' · 'Quantity' · 'InvoiceDate' · 'UnitPrice' · 'CustomerID' · 'Country'
```

```
str(etail_raw)

tibble [541,909 × 8] (S3: tbl_df/tbl/data.frame)
 $ InvoiceNo   : chr [1:541909] "536365" "536365" "536365" "536365" ...
 $ StockCode  : chr [1:541909] "85123A" "71053" "84406B" "84029G" ...
 $ Description: chr [1:541909] "WHITE HANGING HEART T-LIGHT HOLDER" "WHITE METAL LANTERN" "CREAM CUPID HEARTS COAT HANGER" "KNITTED UNION
 $ Quantity   : num [1:541909] 6 6 8 6 6 2 6 6 6 32 ...
 $ InvoiceDate: POSIXct[1:541909], format: "2010-12-01 08:26:00" "2010-12-01 08:26:00" ...
 $ UnitPrice  : num [1:541909] 2.55 3.39 2.75 3.39 3.39 7.65 4.25 1.85 1.85 1.69 ...
 $ CustomerID : num [1:541909] 17850 17850 17850 17850 17850 ...
 $ Country    : chr [1:541909] "United Kingdom" "United Kingdom" "United Kingdom" "United Kingdom" ...
```

```
summary(etail_raw)

      InvoiceNo      StockCode      Description      Quantity
Length   :541909  Length   :541909  Length   :541909  Min.    :-80995.000
N.unique : 25900   N.unique :  4070   N.unique :  4211   1st Qu.:   1.000
N.blank  :      0   N.blank  :      0   N.blank  :      0   Median :   3.000
Min.nchar:      6   Min.nchar:      1   Min.nchar:      1   Mean   :   9.552
Max.nchar:      7   Max.nchar:     12   Max.nchar:     35   3rd Qu.:  10.000
                        NAs       : 1454   Max.    : 80995.000

      InvoiceDate      UnitPrice      CustomerID
Min.    :2010-12-01 08:26:00  Min.    :-11062.060  Min.    :12346
1st Qu.:2011-03-28 11:34:00   1st Qu.:   1.250    1st Qu.:13953
Median :2011-07-19 17:17:00   Median :   2.080    Median :15152
Mean   :2011-07-04 13:34:57   Mean   :   4.611    Mean   :15288
3rd Qu.:2011-10-19 11:27:00   3rd Qu.:   4.130    3rd Qu.:16791
Max.    :2011-12-09 12:50:00   Max.    :38970.000   Max.    :18287
                        NAs       :135080

      Country
Length   :541909
N.unique :   38
N.blank  :      0
Min.nchar:      3
Max.nchar:     20
```

```
cat("Number of rows:", nrow(etail_raw), "\n")
cat("Number of columns:", ncol(etail_raw), "\n")

Number of rows: 541909
Number of columns: 8
```

Missing Value Analysis

Missing values are checked for every column.

`CustomerID` contains missing values in the original UCI Online Retail dataset. Since customer-level analysis is required in this experiment, records without a CustomerID will not be used for the final customer and revenue analysis.

Missing descriptions are also removed because product-level analysis requires a valid product description.

```
missing_values <- colSums(is.na(etail_raw))
missing_values

InvoiceNo:      0 StockCode:      0 Description:      1454 Quantity:      0 InvoiceDate:      0 UnitPrice:      0 CustomerID:      135080 Country:      0

missing_percentage <- colMeans(is.na(etail_raw)) * 100
round(missing_percentage, 2)

InvoiceNo:      0 StockCode:      0 Description:      0.27 Quantity:      0 InvoiceDate:      0 UnitPrice:      0 CustomerID:      24.93 Country:      0
```

Duplicate Record Analysis

Duplicate transaction records can affect revenue calculations. Therefore, duplicate rows are identified and removed during preprocessing.

```
duplicate_count <- sum(duplicated(etail_raw))

cat("Number of duplicate records:", duplicate_count, "\n")

Number of duplicate records: 5268
```

Creating the Transaction Data Source

The transaction table contains:

- InvoiceNo
- StockCode
- CustomerID
- Quantity
- InvoiceDate

This represents the transaction-level information in the retail system.

```
transactions <- retail_raw %>%
  select(
    InvoiceNo,
    StockCode,
    CustomerID,
    Quantity,
    InvoiceDate
  )

head(transactions)
```

A tibble: 6 × 5				
InvoiceNo	StockCode	CustomerID	Quantity	InvoiceDate
<chr>	<chr>	<dbl>	<dbl>	<dtm>
536365	85123A	17850	6	2010-12-01 08:26:00
536365	71053	17850	6	2010-12-01 08:26:00
536365	84406B	17850	8	2010-12-01 08:26:00
536365	84029G	17850	6	2010-12-01 08:26:00
536365	84029E	17850	6	2010-12-01 08:26:00
536365	22752	17850	2	2010-12-01 08:26:00

Creating the Product Data Source

The product table contains:

- StockCode
- Description
- UnitPrice

Each StockCode represents a product identifier.

```
products <- retail_raw %>%
  select(
    StockCode,
    Description,
    UnitPrice
  ) %>%
  distinct(StockCode, .keep_all = TRUE)

head(products)
```

A tibble: 6 × 3		
StockCode	Description	UnitPrice
<chr>	<chr>	<dbl>
85123A	WHITE HANGING HEART T-LIGHT HOLDER	2.55
71053	WHITE METAL LANTERN	3.39
84406B	CREAM CUPID HEARTS COAT HANGER	2.75
84029G	KNITTED UNION FLAG HOT WATER BOTTLE	3.39
84029E	RED WOOLLY HOTTIE WHITE HEART.	3.39
22752	SET 7 BABUSHKA NESTING BOXES	7.65

Creating the Customer Data Source

The customer table contains:

- CustomerID
- Country

Each CustomerID represents a customer.

```
customers <- retail_raw %>%
  select(
    CustomerID,
    Country
  ) %>%
  filter(!is.na(CustomerID)) %>%
  distinct(CustomerID, .keep_all = TRUE)

head(customers)
```

A tibble: 6 × 2

CustomerID	Country
<dbl>	<chr>
17850	United Kingdom
13047	United Kingdom
12583	France
13748	United Kingdom
15100	United Kingdom
15291	United Kingdom

```
cat("Transactions:", nrow(transactions), "rows\n")
cat("Products:", nrow(products), "rows\n")
cat("Customers:", nrow(customers), "rows\n")
```

Transactions: 541909 rows
Products: 4070 rows
Customers: 4372 rows

```
cat("Transaction columns:\n")
names(transactions)

cat("\nProduct columns:\n")
names(products)

cat("\nCustomer columns:\n")
names(customers)
```

Transaction columns:
'InvoiceNo' · 'StockCode' · 'CustomerID' · 'Quantity' · 'InvoiceDate'

Product columns:
'StockCode' · 'Description' · 'UnitPrice'

Customer columns:
'CustomerID' · 'Country'

▼ Data Cleaning: Transactions

The following cleaning operations are performed:

1. Remove duplicate records.
2. Remove records with missing CustomerID.
3. Remove records with missing StockCode.
4. Remove transactions with zero or negative quantities.
5. Convert InvoiceDate to a proper date-time format.

Negative quantities occur in the dataset because some records represent returns or cancellations. For this sales-revenue analysis, only positive quantities are retained.

```
transactions_clean <- transactions %>%
  distinct() %>%
  filter(
    !is.na(CustomerID),
    !is.na(StockCode),
    Quantity > 0
  ) %>%
  mutate(
    InvoiceDate = as.POSIXct(InvoiceDate)
  )
```

```
dim(transactions_clean)
```

392708 · 5

Data Cleaning: Products

Product records are cleaned by:

- Removing missing StockCode values
- Removing missing descriptions
- Removing missing UnitPrice values
- Removing zero or negative UnitPrice values
- Keeping one record per StockCode

Only products with valid positive prices are retained for revenue analysis.

```
products_clean <- products %>%
  mutate(
    UnitPrice = as.numeric(UnitPrice)
  ) %>%
  filter(
    !is.na(StockCode),
    !is.na(Description),
    !is.na(UnitPrice),
    UnitPrice > 0
  ) %>%
  distinct(StockCode, .keep_all = TRUE)
```

```
dim(products_clean)
```

```
3855 · 3
```

▼ Data Cleaning: Customers

Customer records without a CustomerID cannot be uniquely identified. Therefore, records with missing CustomerID are removed.

Duplicate CustomerID records are also removed because the customer table should contain one logical record per customer.

```
customers_clean <- customers %>%
  filter(
    !is.na(CustomerID),
    !is.na(Country)
  ) %>%
  distinct(CustomerID, .keep_all = TRUE)
```

```
dim(customers_clean)
```

```
4372 · 2
```

```
cat("=====  
CLEANING SUMMARY  
=====\n")  
  
cat("Original records:", nrow(retail_raw), "\n")  
cat("Clean transactions:", nrow(transactions_clean), "\n")  
cat("Clean products:", nrow(products_clean), "\n")  
cat("Clean customers:", nrow(customers_clean), "\n")
```

```
=====  
CLEANING SUMMARY  
=====  
Original records: 541909  
Clean transactions: 392708  
Clean products: 3855  
Clean customers: 4372
```

▼ Task 2: Integrate the Multiple Data Sources

Although the supplied UCI dataset is stored in one Excel file, three logical data sources were created from it:

Transactions + Products + Customers

The tables are integrated using:

- `left_join()` between transactions and products using `StockCode`
- `left_join()` between the resulting dataset and customers using `CustomerID`

`left_join()` is selected because transactions are the primary dataset and all valid transactions should initially be preserved while checking for matching product and customer information.

```
transaction_product <- transactions_clean %>%
  left_join(
    products_clean,
    by = "StockCode"
  )
```

```
dim(transaction_product)
```

```
392708 · 7
```

```
head(transaction_product)
```

```
A tibble: 6 × 7
```

InvoiceNo	StockCode	CustomerID	Quantity	InvoiceDate	Description	UnitPrice
<chr>	<chr>	<dbl>	<dbl>	<dtm>	<chr>	<dbl>
536365	85123A	17850	6	2010-12-01 08:26:00	WHITE HANGING HEART T-LIGHT HOLDER	2.55
536365	71053	17850	6	2010-12-01 08:26:00	WHITE METAL LANTERN	3.39
536365	84406B	17850	8	2010-12-01 08:26:00	CREAM CUPID HEARTS COAT HANGER	2.75
536365	84029G	17850	6	2010-12-01 08:26:00	KNITTED UNION FLAG HOT WATER BOTTLE	3.39
536365	84029E	17850	6	2010-12-01 08:26:00	RED WOOLLY HOTTIE WHITE HEART.	3.39
536365	22752	17850	2	2010-12-01 08:26:00	SET 7 BABUSHKA NESTING BOXES	7.65

Checking Unmatched Product Records

After joining transactions with products, records where `Description` or `UnitPrice` is missing indicate that the transaction could not be matched with the product table.

```
unmatched_products <- transaction_product %>%
  filter(
    is.na>Description) |
    is.na>UnitPrice)
  )

cat(
  "Unmatched product records:",
  nrow(unmatched_products),
  "\n"
)
```

```
Unmatched product records: 4831
```

```
retail_data <- transaction_product %>%
  left_join(
    customers_clean,
    by = "CustomerID"
  )
```

```
dim(retail_data)
```

```
392708 · 8
```

```
unmatched_customers <- retail_data %>%
  filter(is.na>Country))

cat(
  "Unmatched customer records:",
  nrow(unmatched_customers),
  "\n"
)
```

```
Unmatched customer records: 0
```

Revenue Calculation

Revenue is calculated for every valid transaction using:

Revenue = Quantity × UnitPrice

This creates a new attribute named `Revenue`.

```
retail_data <- retail_data %>%
  filter(
    !is.na>UnitPrice),
    !is.na>Country)
  ) %>%
  mutate(
    Revenue = Quantity * UnitPrice
  )
```

```
head(retail_data)
```

A tibble: 6 × 9

InvoiceNo	StockCode	CustomerID	Quantity	InvoiceDate	Description	UnitPrice	Country	Revenue
<chr>	<chr>	<dbl>	<dbl>	<dtm>	<chr>	<dbl>	<chr>	<dbl>
536365	85123A	17850	6	2010-12-01 08:26:00	WHITE HANGING HEART T-LIGHT HOLDER	2.55	United Kingdom	15.30
536365	71053	17850	6	2010-12-01 08:26:00	WHITE METAL LANTERN	3.39	United Kingdom	20.34
536365	84406B	17850	8	2010-12-01 08:26:00	CREAM CUPID HEARTS COAT HANGER	2.75	United Kingdom	22.00
536365	84029G	17850	6	2010-12-01 08:26:00	KNITTED UNION FLAG HOT WATER BOTTLE	3.39	United Kingdom	20.34
536365	84029E	17850	6	2010-12-01 08:26:00	RED WOOLLY HOTTIE WHITE HEART.	3.39	United Kingdom	20.34
536365	22752	17850	2	2010-12-01 08:26:00	SET 7 BABUSHKA NESTING BOXES	7.65	United Kingdom	15.30

```
cat("Final number of rows:", nrow(retail_data), "\n")
cat("Final number of columns:", ncol(retail_data), "\n")
```

```
Final number of rows: 387877
Final number of columns: 9
```

```
names(retail_data)
```

```
'InvoiceNo' · 'StockCode' · 'CustomerID' · 'Quantity' · 'InvoiceDate' · 'Description' · 'UnitPrice' · 'Country' · 'Revenue'
```

```
sum(is.na(retail_data))
```

```
0
```

```
sum(duplicated(retail_data))
```

```
0
```

```
summary(retail_data$Revenue)
```

```
   Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
1.000e-03 5.040e+00 1.320e+01 2.772e+01 2.080e+01 1.685e+05
```

Task 3: Sales and Customer Analysis

3.1 Total Sales Revenue

The total sales revenue is calculated by summing the Revenue column across all valid transactions.

```
total_revenue <- retail_data %>%
  summarise(
    Total_Revenue = sum(Revenue)
  )
```

```
total_revenue
```

A tibble: 1 × 1

```
Total_Revenue
```

```
<dbl>
```

```
10752840
```

3.2 Top 5 Products Based on Revenue

Products are grouped using StockCode and Description. Revenue is aggregated for each product and sorted in descending order.

```
top_5_products <- retail_data %>%
  group_by(
    StockCode,
    Description
  ) %>%
  summarise(
    Total_Revenue = sum(Revenue),
    Total_Quantity = sum(Quantity),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Revenue)) %>%
  slice_head(n = 5)
```

```
top_5_products
```

A tibble: 5 × 4

StockCode	Description	Total_Revenue	Total_Quantity
<chr>	<chr>	<dbl>	<dbl>
23843	PAPER CRAFT , LITTLE BIRDIE	168469.60	80995
47566	PARTY BUNTING	142437.56	15283
22423	REGENCY CAKESTAND 3 TIER	135604.80	12384
85123A	WHITE HANGING HEART T-LIGHT HOLDER	93745.65	36763
23166	MEDIUM CERAMIC TOP STORAGE JAR	81032.64	77916

3.3 Top 5 Countries Based on Revenue

Revenue is aggregated at country level to identify the strongest markets.

```
top_5_countries <- retail_data %>%
  group_by(Country) %>%
  summarise(
    Total_Revenue = sum(Revenue),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Revenue)) %>%
  slice_head(n = 5)
```

top_5_countries

A tibble: 5 × 2

Country	Total_Revenue
<chr>	<dbl>
United Kingdom	8861857.1
Netherlands	363884.5
EIRE	331660.2
Germany	263819.0
France	226975.6

3.4 Top 5 Customers Based on Total Purchase Value

Customer purchase value is calculated by summing the revenue generated by each customer.

```
top_5_customers <- retail_data %>%
  group_by(CustomerID) %>%
  summarise(
    Total_Purchase_Value = sum(Revenue),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Purchase_Value)) %>%
  slice_head(n = 5)
```

top_5_customers

A tibble: 5 × 2

CustomerID	Total_Purchase_Value
<dbl>	<dbl>
18102	408760.0
14646	357531.1
17450	186038.0
14911	182690.5
16446	168472.5

3.5 Customer Value Classification

Customers are classified into four categories:

- Low Value
- Medium Value
- High Value
- Premium

Instead of selecting arbitrary monetary thresholds, quartiles are used.

The thresholds are calculated from the distribution of total customer purchase values:

- Q1 → boundary between Low and Medium
- Median → boundary between Medium and High
- Q3 → boundary between High and Premium

The classification is implemented using `case_when()`.

```
customer_value <- retail_data %>%
  group_by(CustomerID) %>%
  summarise(
    Total_Purchase_Value = sum(Revenue),
    .groups = "drop"
  )
```

```
q1 <- quantile(
  customer_value$Total_Purchase_Value,
  0.25,
  na.rm = TRUE
)

median_value <- quantile(
  customer_value$Total_Purchase_Value,
  0.50,
  na.rm = TRUE
)

q3 <- quantile(
  customer_value$Total_Purchase_Value,
  0.75,
  na.rm = TRUE
)

cat("Q1:", q1, "\n")
cat("Median:", median_value, "\n")
cat("Q3:", q3, "\n")
```

```
Q1: 361.005
Median: 802.6
Q3: 2000.64
```

```
customer_value <- customer_value %>%
  mutate(
    Customer_Segment = case_when(
      Total_Purchase_Value <= q1 ~ "Low Value",
      Total_Purchase_Value <= median_value ~ "Medium Value",
      Total_Purchase_Value <= q3 ~ "High Value",
      TRUE ~ "Premium"
    )
  )
```

```
head(customer_value)
```

A tibble: 6 × 3

CustomerID	Total_Purchase_Value	Customer_Segment
<dbl>	<dbl>	<chr>
12346	77183.60	Premium
12347	5438.44	Premium
12348	1790.16	High Value
12349	1926.96	High Value
12350	408.04	Medium Value
12352	1616.57	High Value

```
customer_segments <- customer_value %>%
  group_by(Customer_Segment) %>%
  summarise(
    Number_of_Customers = n(),
    Total_Revenue = sum(Total_Purchase_Value),
    Average_Purchase_Value = mean(Total_Purchase_Value),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Revenue))

customer_segments
```

A tibble: 4 × 4

Customer_Segment	Number_of_Customers	Total_Revenue	Average_Purchase_Value
<chr>	<int>	<dbl>	<dbl>
Premium	1085	8518195.8	7850.8717
High Value	1084	1400834.6	1292.2829
Medium Value	1085	604126.7	556.7988
Low Value	1085	229682.9	211.6893

3.6 Market Performance Analysis

Countries are evaluated based on total revenue.

The country with the highest revenue is considered the high-performing market.

The country with the lowest revenue is considered the underperforming market.

Revenue is used as the primary performance metric because the dataset does not contain cost or profit information.

```
market_performance <- retail_data %>%
  group_by(Country) %>%
  summarise(
    Total_Revenue = sum(Revenue),
    Number_of_Customers = n_distinct(CustomerID),
    Number_of_Transactions = n(),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Revenue))

market_performance
```

A tibble: 37 × 4

Country	Total_Revenue	Number_of_Customers	Number_of_Transactions
<chr>	<dbl>	<int>	<int>
United Kingdom	8861857.13	3921	344847
Netherlands	363884.48	9	2340
EIRE	331660.17	3	7155
Germany	263818.97	94	8934
France	226975.60	87	8196
Australia	173918.61	9	1242
Spain	67426.09	28	2396
Switzerland	66619.97	20	1815
Japan	48600.22	8	321
Belgium	47858.02	24	1981
Sweden	43652.09	8	447
Norway	40283.20	10	1067
Portugal	32679.45	19	1430
Finland	23306.79	12	671
Channel Islands	22059.72	9	738
Denmark	21291.79	8	393
Italy	21065.45	14	751
Cyprus	16829.62	7	647
Singapore	11497.98	1	217
Poland	10264.54	6	328
Austria	9883.01	9	367
Israel	9088.79	3	242
Greece	6128.80	4	145
Iceland	5438.44	1	182
Canada	4315.90	4	151
USA	4138.83	4	178
Unspecified	2952.93	4	239
Malta	2890.34	2	112
United Arab Emirates	2747.72	2	68
Lebanon	2085.13	1	45
Lithuania	1804.34	1	34
European Community	1575.65	1	60
Brazil	1225.35	1	32
RSA	1090.43	1	55
Czech Republic	1026.26	1	25
Bahrain	716.78	2	17
Saudi Arabia	181.44	1	9

```
high_performing_market <- market_performance %>%
  slice_max(
    order_by = Total_Revenue,
    n = 1
  )

underperforming_market <- market_performance %>%
  slice_min(
    order_by = Total_Revenue,
    n = 1
  )

high_performing_market
underperforming_market
```

A tibble: 1 × 4

Country	Total_Revenue	Number_of_Customers	Number_of_Transactions
<chr>	<dbl>	<int>	<int>
United Kingdom	8861857	3921	344847

A tibble: 1 × 4

Country	Total_Revenue	Number_of_Customers	Number_of_Transactions
<chr>	<dbl>	<int>	<int>
Saudi Arabia	181.44	1	9

```
cat(
  "High-performing market:",
  high_performing_market$Country,
  "\n"
)

cat(
  "Revenue:",
  round(high_performing_market$Total_Revenue, 2),
  "\n\n"
)

cat(
  "Underperforming market:",
  underperforming_market$Country,
  "\n"
)

cat(
  "Revenue:",
  round(underperforming_market$Total_Revenue, 2),
  "\n"
)
```

High-performing market: United Kingdom
Revenue: 8861857

Underperforming market: Saudi Arabia
Revenue: 181.44

Market Interpretation

The high-performing market is the country with the highest total revenue. Its strong revenue contribution indicates a relatively strong sales presence in the dataset.

The underperforming market has the lowest total revenue. This could be due to fewer customers, fewer transactions, lower purchase frequency or lower average transaction value.

Therefore, revenue should be considered together with customer count and transaction count when making strategic decisions.

Task 4: Store and Retrieve Data Using SQL

The final cleaned and integrated dataset is stored in a SQLite database.

Database: retail_sales.db

Table: retail_sales

```
db_path <- "/content/retail_sales.db"

con <- dbConnect(
  SQLite(),
  db_path
)
```

```
dbWriteTable(
  con,
  "retail_sales",
  retail_data,
  overwrite = TRUE
)
```

```
dbListTables(con)
```

```
'retail_sales'
```

```
dbGetQuery(
  con,
```

```
"SELECT COUNT(*) AS Total_Records
FROM retail_sales"
)
```

```
A data.frame: 1 ×
1
Total_Records
<int>
387877
```

```
dbGetQuery(
  con,
  "SELECT *
FROM retail_sales
LIMIT 10"
)
```

A data.frame: 10 × 9

InvoiceNo	StockCode	CustomerID	Quantity	InvoiceDate	Description	UnitPrice	Country	Revenue
<chr>	<chr>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<dbl>
536365	85123A	17850	6	1291191960	WHITE HANGING HEART T-LIGHT HOLDER	2.55	United Kingdom	15.30
536365	71053	17850	6	1291191960	WHITE METAL LANTERN	3.39	United Kingdom	20.34
536365	84406B	17850	8	1291191960	CREAM CUPID HEARTS COAT HANGER	2.75	United Kingdom	22.00
536365	84029G	17850	6	1291191960	KNITTED UNION FLAG HOT WATER BOTTLE	3.39	United Kingdom	20.34
536365	84029E	17850	6	1291191960	RED WOOLLY HOTTIE WHITE HEART.	3.39	United Kingdom	20.34
536365	22752	17850	2	1291191960	SET 7 BABUSHKA NESTING BOXES	7.65	United Kingdom	15.30
536365	21730	17850	6	1291191960	GLASS STAR FROSTED T-LIGHT HOLDER	4.25	United Kingdom	25.50
536366	22633	17850	6	1291192080	HAND WARMER UNION JACK	1.85	United Kingdom	11.10
536366	22632	17850	6	1291192080	HAND WARMER RED POLKA DOT	1.85	United Kingdom	11.10
536367	84879	13047	32	1291192440	ASSORTED COLOUR BIRD ORNAMENT	1.69	United Kingdom	54.08

SQL Query 1: Top 5 Customers by Revenue

The following SQL query calculates the total purchase value of each customer and returns the five customers with the highest revenue.

```
query_1 <- "
SELECT
  CustomerID,
  SUM(Revenue) AS Total_Revenue
FROM retail_sales
GROUP BY CustomerID
ORDER BY Total_Revenue DESC
LIMIT 5;
"

top_customers_sql <- dbGetQuery(
  con,
  query_1
)

top_customers_sql
```

A data.frame: 5 × 2

CustomerID	Total_Revenue
<dbl>	<dbl>
18102	408760.0
14646	357531.1
17450	186038.0
14911	182690.5
16446	168472.5

SQL Query 2: Total Revenue by Country

The following query calculates total sales revenue for each country and orders the markets from highest to lowest revenue.

```
query_2 <- "
SELECT
  Country,
```

```
SUM(Revenue) AS Total_Revenue
FROM retail_sales
GROUP BY Country
ORDER BY Total_Revenue DESC;
"

revenue_country_sql <- dbGetQuery(
  con,
  query_2
)

head(revenue_country_sql, 10)
```

A data.frame: 10 × 2

	Country	Total_Revenue
	<chr>	<dbl>
1	United Kingdom	8861857.13
2	Netherlands	363884.48
3	EIRE	331660.17
4	Germany	263818.97
5	France	226975.60
6	Australia	173918.61
7	Spain	67426.09
8	Switzerland	66619.97
9	Japan	48600.22
10	Belgium	47858.02

Additional SQL Query: Overall Retail Performance

This query retrieves the total revenue, number of transactions and number of unique customers from the SQLite database.

```
query_3 <- "
SELECT
  SUM(Revenue) AS Total_Revenue,
  COUNT(*) AS Total_Transactions,
  COUNT(DISTINCT CustomerID) AS Total_Customers
FROM retail_sales;
"

dbGetQuery(
  con,
  query_3
)
```

A data.frame: 1 × 3

Total_Revenue	Total_Transactions	Total_Customers
<dbl>	<int>	<int>
10752840	387877	4339

SQLite Table Structure

The database schema is inspected using SQLite's `PRAGMA` command.

```
dbGetQuery(
  con,
  "PRAGMA table_info(retail_sales)"
)
```

A data.frame: 9 × 6

cid	name	type	notnull	dflt_value	pk
<int>	<chr>	<chr>	<int>	<lgl>	<int>
0	InvoiceNo	TEXT	0	NA	0
1	StockCode	TEXT	0	NA	0
2	CustomerID	REAL	0	NA	0
3	Quantity	REAL	0	NA	0
4	InvoiceDate	REAL	0	NA	0
5	Description	TEXT	0	NA	0
6	UnitPrice	REAL	0	NA	0
7	Country	TEXT	0	NA	0
8	Revenue	REAL	0	NA	0

```
dbDisconnect(con)
```

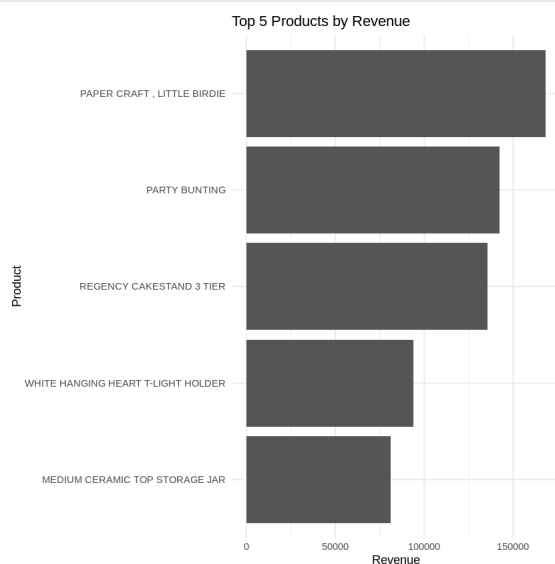
```
file.exists("/content/retail_sales.db")
```

```
TRUE
```

▼ Data Visualization

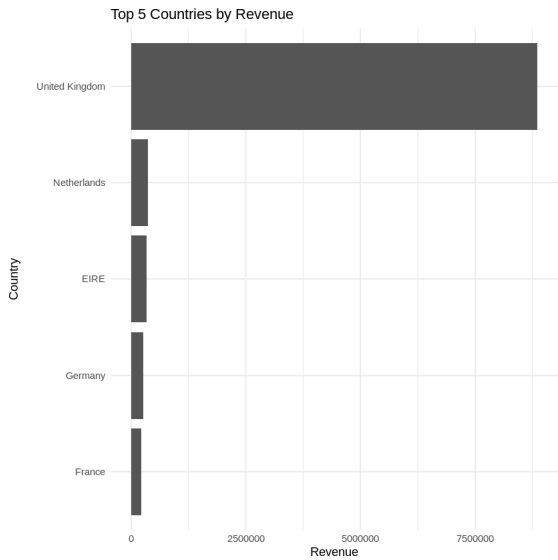
Visualizations are created to better understand product, market and customer performance.

```
ggplot(  
  top_5_products,  
  aes(  
    x = reorder(Description, Total_Revenue),  
    y = Total_Revenue  
  )  
) +  
  geom_col() +  
  coord_flip() +  
  labs(  
    title = "Top 5 Products by Revenue",  
    x = "Product",  
    y = "Revenue"  
  ) +  
  theme_minimal()
```

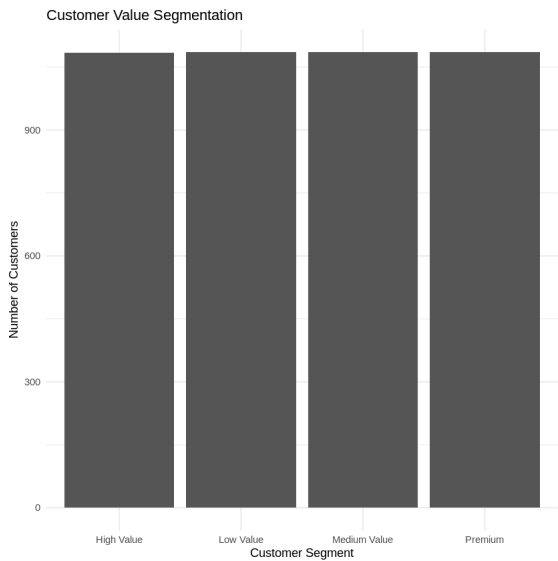


```
ggplot(  
  top_5_countries,  
  aes(  
    x = reorder(Country, Total_Revenue),  
    y = Total_Revenue  
  )  
) +  
  geom_col() +  
  coord_flip() +  
  labs(  
    title = "Top 5 Countries by Revenue",  
    x = "Country",  
    y = "Revenue"  
  )
```

```
) +  
theme_minimal()
```



```
ggplot(  
  customer_segments,  
  aes(  
    x = Customer_Segment,  
    y = Number_of_Customers  
  )  
) +  
  geom_col() +  
  labs(  
    title = "Customer Value Segmentation",  
    x = "Customer Segment",  
    y = "Number of Customers"  
  ) +  
  theme_minimal()
```



Final Validation

The following checks verify that the final dataset is ready for analysis and database storage.

```
cat("===== FINAL VALIDATION =====\n")  
  
cat(  
  "Final Rows:",  
  nrow(retail_data),  
  "\n"  
)  
  
cat(  
  "Final Columns:",  
  ncol(retail_data),  
  "\n"  
)  
  
cat(  
  "Missing Values:",
```



```
    sum(is.na(retail_data)),  
    "\n"  
  )  
  
  cat(  
    "Duplicate Rows:",  
    sum(duplicated(retail_data)),  
    "\n"  
  )  
  
  cat(  
    "Total Revenue:",  
    round(sum(retail_data$Revenue), 2),  
    "\n"  
  )  
  
  cat(  
    "Unique Customers:",  
    n_distinct(retail_data$CustomerID),  
    "\n"  
  )
```